

DBMS COURSE PROJECT

A relational database in MySQL for alumni, mentoring, events and donations

K. Harshitha | 25WU0102134

B.Tech CSE-AIML, II Year | Woxsen University, Hyderabad

Course 25TU03MJM0 | Faculty: Kiranmayee Adavala | 2026-27

Alumni

Student

Mentor

Mentorship

Event

Event_Registration

Engagement

Donation

Why a database: problem and proposed system

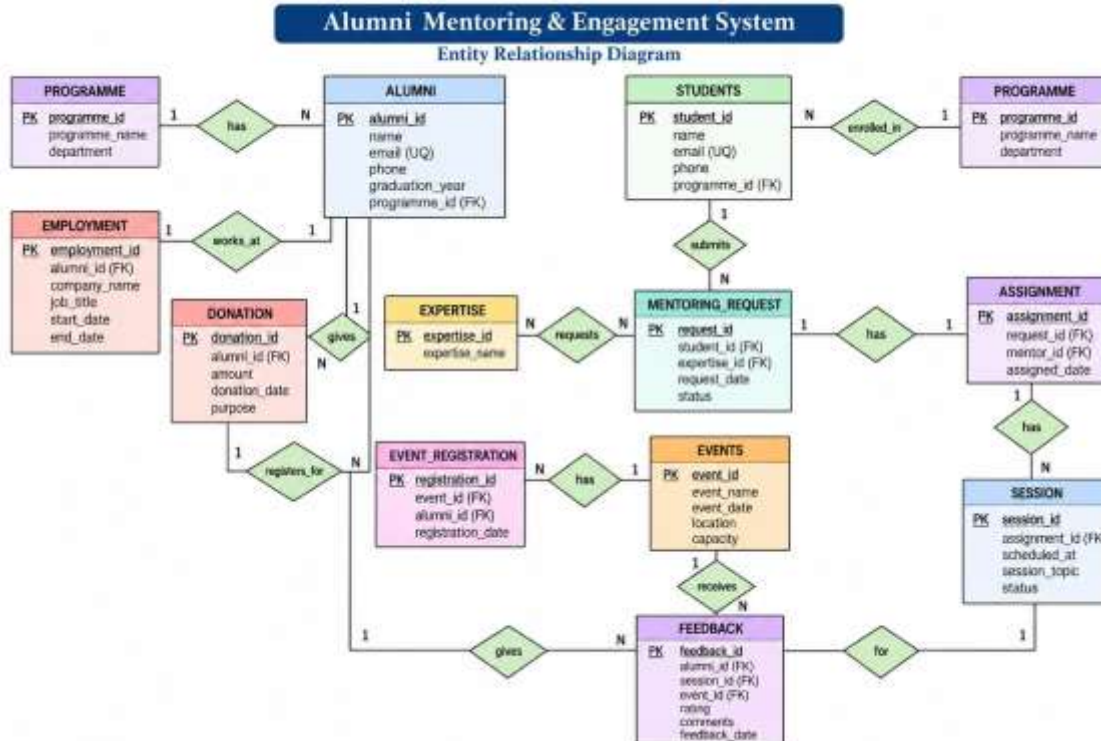
Problem

- Alumni data sits in scattered files and chats
- Mentors are hard to find by expertise
- Events and engagement are not tracked consistently
- Duplicates and manual reports hurt reliability

Proposed system

- MySQL 8.x with 8 related tables, joined by primary and foreign keys
- Python + Tkinter UI for insert, delete and view
- Normalized to 3NF to cut duplication
- SQL for DDL, DML, joins and aggregation

ER diagram and key relationships



Alumni to Mentor

An alumnus can act as a mentor

Student, Mentor, Mentorship

One mentor guides many students over time

Alumni, Event, Registration

Alumni register for many events

Alumni to Engagement, Donation

One alumnus, many records

Relational schema and normalization

Table	Keys
ALUMNI	PK AlumniID, UQ Email
STUDENT	PK StudentID, UQ Email
MENTOR	PK MentorID, FK AlumniID
MENTORSHIP	PK, FK StudentID, MentorID
EVENT	PK EventID
EVENT_REGISTRATION	PK, FK EventID, AlumniID
ENGAGEMENT	PK, FK AlumniID
DONATION	PK, FK AlumniID

1NF

Atomic values; repeating groups moved to MENTORSHIP and EVENT_REGISTRATION

2NF

Attributes depend on the whole key

3NF

Attributes stored with the entity they describe

Constraints: PK, FK, UNIQUE, NOT NULL, CHECK

SQL in action: DDL, DML and a join query

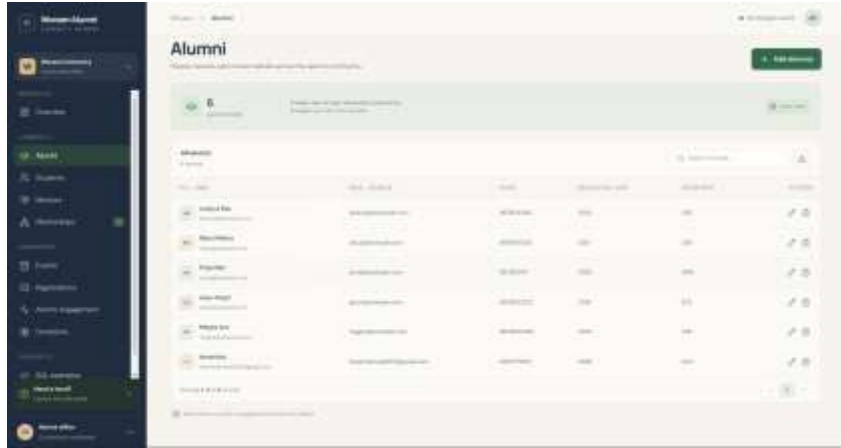
```
CREATE TABLE Mentor (  
  MentorID INT PRIMARY KEY AUTO_INCREMENT,  
  AlumniID INT NOT NULL,  
  Expertise VARCHAR(150) NOT NULL,  
  FOREIGN KEY (AlumniID)  
    REFERENCES Alumni(AlumniID)  
);  
INSERT INTO Mentor  
  (AlumniID, Expertise)  
VALUES (1, 'Machine Learning');
```

```
SELECT s.Name AS Student,  
       a.Name AS Mentor, m.Status  
FROM Mentorship m  
JOIN Student s ON m.StudentID = s.StudentID  
JOIN Mentor mt ON m.MentorID = mt.MentorID  
JOIN Alumni a ON mt.AlumniID = a.AlumniID  
WHERE m.Status = 'Active';
```

Student	Mentor	Status
Aarav Kumar	Ananya Rao	Active

Also demonstrated: filtering with LIKE, GROUP BY counts, UPDATE and DELETE. Check dependent rows before deleting.

User interface and implementation

[View records](#)

Insert record

- Stack: MySQL, Python 3, MySQL Connector, Tkinter, Git + GitHub
- Flow: connect, pick module, submit form, run INSERT / DELETE / SELECT, refresh to verify

Testing and conclusion

- 10 test cases: insert, view, delete, duplicate email, blank field, bad foreign key, event registration, filtering, connection
- The design demonstrates keys, 3NF, DDL/DML, joins and a working UI

Future enhancements

- Role-based login
- Mentor recommendation by expertise
- Email reminders and dashboards
- CSV import/export and audit logs
- Secure web deployment